TOML Projects: Data-driven techniques for IoT monitoring networks.

Jose M. Barcelo Ordinas, Jorge Garcia Vidal

Universidad Politècnica de Catalunya (UPC-BarcelonaTECH),
Computer Architecture Dept.
jose.maria.barcelo@upc.edu
jorge.garcia@upc.edu

October 21, 2026

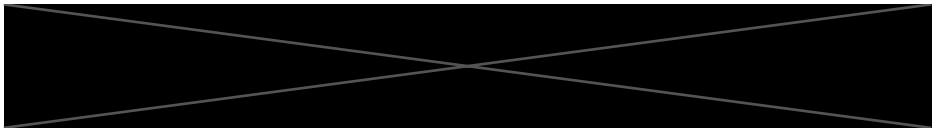
This project consists of two reports submitted in PDF format. The two reports are related to improving data quality in heterogeneous IoT monitoring networks where high- and low-cost sensors are deployed.

- **Report 1: Denoising of low-cost sensors and creating virtual sensors.** First, the main objective is to denoise a low-cost sensor using the 'eigenfaces' technique. Second, we will create a virtual sensor using graph signal processing techniques.
- **Report 2: Development of a proxy sensor for unregulated pollutants.** In this case, the objective is to explore and compare various non-linear machine learning models to develop a proxy sensor for an unregulated pollutant, such as black carbon, using low-cost regulated gas sensors and meteorological variables.

The PDF format of each report must have the following characteristics: a font size of 11 or 12 points, single line spacing, a single column, and a maximum length of between 20 and 25 pages. The report must follow the format of a full-length paper: introduction (rationale), related work (find articles in the bibliography relating to the topic), a brief explanation of the theory you will be using (do not explain everything, but indicate the models, architectures, and procedure you will be employing), results (plots, figures, tables, and an explanation of these), and conclusions.

The evaluation of the two reports will include a face-to-face interview. The grading of each report will be weighted considering the following aspects:

- 20% for the report's format: The report should include an abstract, an introduction motivating the problem, a related work section, and a short section describing the models and methodology. Then, a short section for the dataset description. Finally, a section with results and a section with conclusions. The grading of this part is related to how well it is written and organized, and how clear it is.
- 40% for the quality of the results, i.e., which results are shown and how they are presented (e.g., plots, tables). You have to decide which results to show, which metrics, and in which format.
- 40% for the quality of the explanation and conclusions of the results, i.e., how the results are explained and what conclusions you get.



Some tips on how to solve convex optimization problems in Python, plot graphs, or other useful tools. There are several solvers to solve optimization problems in Python.

CVXPY[1] toolbox is also a package for convex optimization based on the Python programming language. Look at the tutorial[2] on how to create variables or solve a problem with parameters (regularization). In general, follow the guide[3] with examples of use. Optimizing traces of matrices is a semidefinite program (SDP), so we recommend that you look at the tutorials of cvxpy related to SDP's[4][5].

CVXPY is not the only tool that you can use to solve optimization problems; you have other options to solve optimization problems.

For example, you can also use **Scipy**[6] and in particular its optimization module[7]. The **minimize**[8] module allows you to solve many optimization problems (non-linear), which, by default, use the SLSQP solver[9], which applies Newton's method as a basis for the KKT conditions.

CVXOPT[10] toolbox is a package for convex optimization based on the Python

[1] https://www.cvxpy.org/
[2] https://www.cvxpy.org/tutorial/intro/index.html
[3] https://www.cvxpy.org/tutorial/index.html
[4] https://www.cvxpy.org/tutorial/constraints/index.html#sparsity-attribute, look at Semidefinite matrices, where it tells you how to specify a positive definite matrix.
[5] https://www.cvxpy.org/version/1.1/examples/basic/sdp.html
[6] https://docs.scipy.org/doc/scipy/reference/index.html
[7] https://docs.scipy.org/doc/scipy/reference/optimize.html
[8] https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.minimize.html#scipy.optimize.minimize
[9] https://en.wikipedia.org/wiki/Sequential_quadratic_programming
[10] https://cvxopt.org/

programming language. The library is optimized to work with convex optimization problems. It is interesting as a tool, although not easy to use. CVXPY is easier to use than CVXOPT, although it is not as powerful. Thus, if you are familiar with CVXOPT you can use it.

1 Report 1: Denoising of low-cost sensors and creation of virtual sensors.

Deadline: to be delivered last day of class (18th of december 2026) using "practicals" in RACO. Interview in january.

1.1 Denoising of low-cost sensors

The objective of this project is to perform denoising for LCSs used in IoT monitoring platforms. Sensors are first calibrated in situ using linear or nonlinear machine learning models that only take into account instantaneous measurements. We present calibrated data using a linear model (multiple linear regression, MLR) and a nonlinear model (support vector regression, SVR), Figure 1.

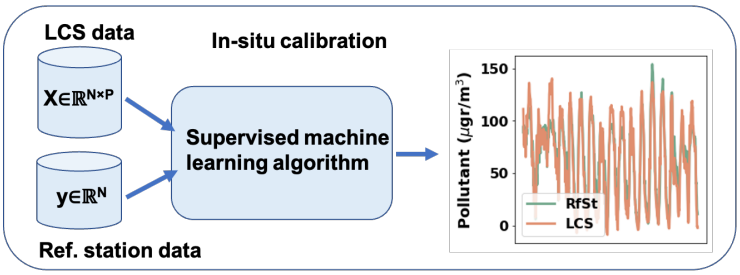


Figure 1: Calibration using a supervised machine learning mechanism.

To improve the estimation, i.e., to decrease the RMSE value, in this second stage, we propose to use the temporal daily correlations of reference instrumentation. To do this, calibrated sensor data is taken at a temporal granularity, in this case daily, to take advantage of the temporal relationships of the data. The temporal pattern-based denoising (TPB-D) approach, Figure 2, consists of a denoising step based on a subspace generated by the singular value decomposition, in which the in-situ calibrated LCS data are projected onto the left-singular vector space obtained from a database of data collected by reference instruments daily.

The denoised data can be further improved if the projected sensor signal is recal-

ibrated by a linear mapping onto the left singular vector space of the reference data projection, the TPB-C mechanism in the Figure. However, we will not do it on this homework, and we will only perform the denoising step.

The theory for solving this task can be found in section 5 of the lecture notes on "linear algebra" and is summarized in section 1.3 of this document.

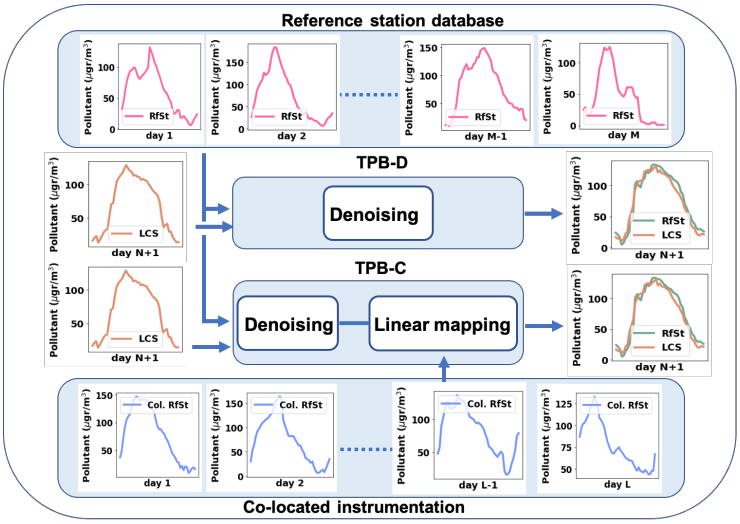


Figure 2: Denoising using a singular value decomposition (SVD).

1.1.1 Project Realization.

The data consists of a CSV file called "Report-1-Denoising.csv", the format being as follows:

date, RefSt, Sensor_MLR_O3, Sensor_SVR_O3

where it can be seen that the first row is a header that describes the data:

- **date:** Timestamp (UTC) for each measurement,
- **RefSt:** Reference Station O_3 concentrations, in μgr/m^3.
- **Sensor_MLR_O3:** Calibrated O_3 concentrations using MLR, in μgr/m^3,
- **Sensor_SVR_O3:** Calibrated O_3 concentrations using SVR, in μgr/m^3

1.2 Visualize your data.

The first step consists of understanding the data. For that purpose, the best approach is to plot several curves to see the temporal behaviour of your sensor

data. We recommend using PANDAS as a tool to handle the data.

Second, you can compare the calibration quality in situ by calculating the RMSE and R^2 of the estimated data for MLR and SVR against the data for the reference station.

Finally, organize your data into days (remove those days that are not complete). In the report, mention how many days you get. Again, try to visualize your daily data and plot several days in the report comparing the daily reference station data and the daily sensor estimated data (both with MLR and SVR).

1.3 Denoise the data.

Now you have to denoise the data using the eigenfaces methodology as lectured in class. Here, the daily data are equivalent to the faces. The face database is now a **reference station** daily data database and the sensor data are the faces you want to denoise. You have to obtain the best **reference station** subspace using the RMSE as a metric to see which is the best dimension, e.g., plot a curve with x-axis the rank of the subspace and y-axis the RMSE obtained by reconstructing the LCS daily signals with each rank. Compare the RMSE with the one obtained in the calibration. For example, for each $\hat{\mathbf{x}}_i$ LCS daily data, you can reconstruct it using:

$$\tilde{\mathbf{x}}_i = \mathbf{\Psi} + \mathbf{U}_\kappa \mathbf{U}_\kappa^T \hat{\mathbf{x}}_i$$

where $\hat{\mathbf{x}}_i$=$\mathbf{x}_i$−$\mathbf{\Psi}$, $\mathbf{\Psi}$ is the average of daily data of the reference station, and $\mathbf{U}$ comes from the SVD of $\mathbf{Y}$ after subtracting the average reference station daily data.

Repeat the process using the Gavish *et al.* method. This method consists of obtaining an approximation for the low-rank value κ in the absence of a reference instrument in place. Gavish *et al.* propose the recovery of low-rank matrices by introducing singular value thresholds. Assuming that we want to recover the data using the centered matrix $\hat{\mathbf{Y}}_R$=$[\hat{\mathbf{y}}_{R_1}, \ldots, \hat{\mathbf{y}}_{R_M}]$∈$\mathbb{R}^{D\times M}$, where each $\hat{\mathbf{y}}_{R_i}$ is a day of data from the reference station, the singular threshold value is given as:

$$\tilde{\sigma}=w(\beta) * \sigma_{med} \tag{1.3.1}$$

where σ_{med} is the median of the singular values, while $w(\beta)$ is obtained as:

$$w(\beta)\approx 0.56\beta^3 - 0.95\beta^2 + 1.82\beta + 1.43 \tag{1.3.2}$$

where β=D/M, with D the dimension of the daily data and M the number of days used by the nearby reference stations. Finally, κ corresponds to the number of singular values that are greater than the threshold $\tilde{\sigma}$.

Draw a curve of the singular values and which is the κ value obtained with the formula, and compare it with the one using the optimization you used previously.

Repeat the result of the denoising step, obtaining the RMSE using the Gavish *et al.* κ value and comparing it with the previous RMSE obtained with the experimental κ value.

1.4 Creating virtual sensors

The idea behind this section of the paper is taken from our article [1]. The goal of this paper [1] is to present some well-known and novel graph tools to find a good representation of sensor networks using structured data for IoT networks. There are several ways to define the graph that represents the network topology.

The classical method proposed in the literature for this type of sensor network consists of creating the edges that join the nodes of the graph following a function that depends on the distance between the nodes. This method has the disadvantage of not taking into account the similarity of the measurements taken by the sensors. Thus, one of the challenges and key points for the representation of a sensor network using graphs is to estimate the structure of the graph underlying the measured data. There are several ways to construct the graph from the data, depending on whether statistical methods based on graphical probabilistic models, such as Markov random fields (MRFs) like Graphical Lasso (GLASSO), or methods based on graph signal processing (GSP) that obtain the Laplacian matrix to represent the topology of the graph are used.

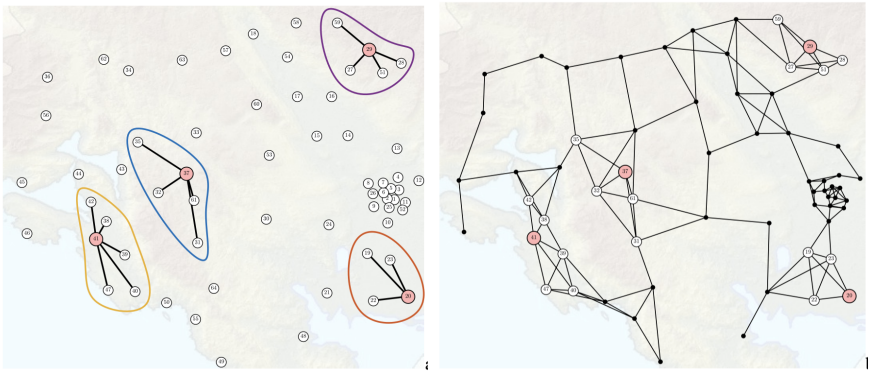
In this task, we are going to explore 3 different ways to construct the graph in a small 12-node sensor network: i) a distance-based model, ii) a GLASSO method, and iii) a GSP-based method, in which we will obtain the Laplacian matrix.

1.4.1 Motivation

The graph signal processing (GSP) framework was conceived in the last decade with the ambition to generalize the tools of classical digital signal processing to the case where the signal is defined over an irregular structure modeled by a graph.

Let us take the example (taken from Ljubisa Stankovic's *et al.* survey paper, "Understanding the basis of graph signal processing via an intuitive example-driven approach", arXiv, May 2019) in which a set of temperature sensors is deployed over a large region. We are interested, Figure 3.a), in finding the local neighborhood of the nodes, so that we can find a graph representing the network, Figure 3.b).

We have been able to connect (we will see later how), for example, node 20 with nodes 19, 22, 23, and node 29 with nodes 27, 28, 51, 59, etc. The objective is that now we can consider that the signal in a given node n is related to its neighborhood. In other words, we can create a virtual sensor at the location



(a) Local neighborhood of single nodes. (b) Local neighborhood for all sensors.

Figure 3: Multisensor IoT example, where nodes measure temperature (Figure taken from L. Stankovic et al. "Understanding the Basis of Graph Signal Processing via an Intuitive Example-Driven Approach" paper). Graph representation.

of n. A virtual sensor measures the signal at the location without placing a physical sensor. Then, the signal at position n is reconstructed, using some machine learning method, from the neighboring nodes. One way of representing the relationships between nodes is to use the adjacency matrix, $\boldsymbol{A}$, defined as a matrix in which ones appear in the positions where two nodes are connected and zeros in the positions where the nodes are not connected at all.

$$\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x} \tag{1.4.1}$$

For example, let us consider a network with five nodes. The matrix indicates that the data of node 1 (first row of the matrix $\boldsymbol{A}$) are correlated with those of nodes 2 and 4; the data of node 2 (second row), with those of nodes 1 and 5; the data of node 3 (third row), with those of nodes 4 and 5; the data of node 4 (fourth row) with that of nodes 1 and 3; and the data of node 5 (fifth row) with that of nodes 2 and 3.

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{bmatrix} \tag{1.4.2}$$

GSP aims to provide a framework capable of operating on a graph of this type, making use of these relationships.

1.4.2 Adjacency matrix, the weighted matrix and the Laplacian matrix

The adjacency matrix is not the only way to represent the network. Other ways are the use of a weighted matrix $\boldsymbol{W}$ and the Laplacian matrix $\boldsymbol{L}$.

The weighted matrix $\boldsymbol{W}$ has coefficients $w_{ij}>0$ if node i is connected to node j and zero otherwise. The idea behind the weighted matrix is that the cost of the edges between connected nodes is not equal to one, thus considering that there are nodes better connected than others. In this case:

$$y(n) = \sum_{m \in N(n)} w_{nm} x(m) \longrightarrow \boldsymbol{y} = \boldsymbol{W}\boldsymbol{x} \tag{1.4.3}$$

We can observe that the operator $\boldsymbol{A}$ is a special case of the operator $\boldsymbol{W}$, in which all weights are considered to be of the same value and equal to 1.

Finally, we can use a third operator called the Laplacian matrix $\boldsymbol{L}$, which is obtained as:

$$\boldsymbol{L} = \boldsymbol{D} - \boldsymbol{W} \tag{1.4.4}$$

where $\boldsymbol{D}$ is the degree matrix, and it has coefficients on the diagonal $d_{ii} = \sum_{j \neq i} w_{ij}$ (sum of the row except the value on the diagonal) and the rest are 0.

Let us see how to obtain the Laplacian matrix $\boldsymbol{L}$, from the weight matrix $\boldsymbol{W}$:

$$\boldsymbol{W} = \begin{bmatrix} 0 & .6 & .3 & 0 \\ .6 & 0 & .1 & .4 \\ .3 & .1 & 0 & .2 \\ 0 & .4 & .2 & 0 \end{bmatrix} \tag{1.4.5}$$

We first obtain the diagonal matrix $\boldsymbol{D}$ as the sum of rows $d_{ii} = \sum_{j \neq i} w_{ij}$:

$$\boldsymbol{D} = \begin{bmatrix} .9 & 0 & 0 & 0 \\ 0 & 1.1 & 0 & 0 \\ 0 & 0 & .6 & 0 \\ 0 & 0 & 0 & .6 \end{bmatrix} \tag{1.4.6}$$

Then, the Laplacian will be:

$$\boldsymbol{L} = \boldsymbol{D} - \boldsymbol{W} = \begin{bmatrix} .6 & -.6 & -.3 & 0 \\ -.6 & 1.1 & -.1 & -.4 \\ -.3 & -.1 & .6 & -.2 \\ 0 & -.4 & -.2 & 0.6 \end{bmatrix} \tag{1.4.7}$$

1.4.3 Creating the graph

Sometimes there is intrinsic knowledge of the weights (physical knowledge of the weights), for example, in circuits of electronic systems, social networks, etc.

With this knowledge, the weights can be selected or defined. However, at other times, we have to rely on other approaches. Let us describe three ways to create the graph (matrices $\boldsymbol{W}$ or $\boldsymbol{L}$, since matrix $\boldsymbol{A}$ is a special case of matrix $\boldsymbol{W}$) of an IoT monitoring network. Consider a network with N nodes (sensors), each measuring M samples.

- **Geometry of the vertex:** If we know the geo-locations of the nodes, e.g., latitude and longitude, we can use the haversine function[11] to obtain the Euclidean distances between pairs of nodes. Now, using the distances d_{ij} between each pair of nodes i and j, we can obtain a weight for that pair of nodes using a decreasing function of distance, e.g., a Radial Basis Function (RBF):

$$w_{ij} = \begin{cases} e^{-d_{ij}/(2\theta)} & \text{if } d_{ij}<T, \\ 0 & \text{otherwise.} \end{cases} \tag{1.4.8}$$

  with $i, j= 1, 2, \ldots,$ N, and if $i=j$, then $w_{ij}=0$. The choice of the parameters T (threshold) and θ (standard deviation of the distribution) and of how to use the metric (in this case, inside a Gaussian distribution) is dictated by the application and by the analyst's experience.

- **Obtain the Weighted matrix W from the data measured matrix $\boldsymbol{X}$ using GMRF (Gaussian Markov Random Fields).**

  A first example is to obtain the weight matrix using the correlations between sensor data, using undirected graphical models, also known as Markov random fields (MRFs), in which local neighborhoods of the graph capture the independence structure of the variables. For that purpose, we consider that the signals of each node $\boldsymbol{x_1}, \ldots, \boldsymbol{x_N}$ (with $\boldsymbol{x_i}\in \mathrm{R}^M$) form a multivariate Gaussian distribution N(0, $\boldsymbol{\Sigma}$). The weighted matrix is then obtained from the precision matrix $\boldsymbol{\Theta}= \boldsymbol{\Sigma^{-1}}$. Since only the sample covariance is known, the graphical Lasso problem (GLASSO) [2] is to estimate the precision matrix $\bar{\boldsymbol{\Theta}}$:

$$\begin{aligned} &\text{minimize} && \operatorname{tr}(S\Theta) - \log\det(\Theta) + \lambda \sum_{j\neq k} |\Theta_{jk}| \\ &\text{subject to} && \boldsymbol{\Theta} \geq 0 \\ &\text{variable} && \boldsymbol{\Theta} \end{aligned} \tag{1.4.9}$$

  where $\boldsymbol{S}$ is the sample covariance of $\boldsymbol{X}$ and λ is the penalizing parameter. Now, we define the weight matrix as $\boldsymbol{W}=\hat{\boldsymbol{\Theta}}$. We can control the sparsity of the graph by fixing a threshold T: if $\hat{\boldsymbol{\Theta}}_{\boldsymbol{ij}}\leq T$ then $\hat{\boldsymbol{\Theta}}_{\boldsymbol{ij}}=0$. Thus, these links with values $\hat{\boldsymbol{\Theta}}_{\boldsymbol{ij}}=0$ are disconnected in the graph.

[11] The haversine formula is a very accurate way of computing distances between two points on the surface of a sphere using the latitude and longitude of the two points. You have a Python function in scikit-learn documentation called haversine_distances and another one in PyPI called haversine.

The GLASSO problem is a convex optimization problem, since the penalty l-1 is convex. GLASSO is a type of SDP (Semidefinite Programming) since the variable (the precision matrix) is a semidefinite matrix. In general, interior point methods should be able to solve it if the dimension of the matrix is small. For large matrices, solvers use the GLASSO algorithm, which consists of decomposing the dual problem defined in 1.4.9 into a series of subproblems and iteratively solving the subproblems until convergence. You can use the Python library *sklearn* for GLASSO due to the complexity of programming from scratch this approach.

- **Obtain the Laplacian matrix L from the measured data matrix $\boldsymbol{X}$.**

  A third way to obtain the graph is to use a non-linear optimization model to directly obtain the Laplacian matrix $\boldsymbol{L}$ from the measured data matrix $\boldsymbol{X}$. The method, proposed by Dong et al. [3], proposes an optimization problem for the Laplacian matrix under the assumption that the signal is smooth. A smoothness model assumes that the signal takes similar values at neighboring vertices. An example of a smooth signal model could be the temperature observed at different locations in a flat geographical region. The measure of smoothness of a signal $\boldsymbol{x}$ on the graph G is usually defined by the so-called Laplacian quadratic form:

$$Q(\boldsymbol{L}) = \boldsymbol{x}^\top \boldsymbol{L}\boldsymbol{x} = \sum_{ij} w_{ij}(x(i) - x(j))^2 \tag{1.4.10}$$

  where w_{ij} are the weights of the weight matrix $\boldsymbol{W}$ and $\boldsymbol{L}$ is the Laplacian matrix. Clearly, $Q(\boldsymbol{L})= 0$ when $\boldsymbol{x}$ is a constant signal on the graph, that is, a signal without variation (smooth). Then, the smaller the $Q(\boldsymbol{L})$ term, the smoother the signal. A natural criterion is to learn a graph (or, equivalently, its Laplacian matrix $\boldsymbol{L}$) such that the signal variation on the resulting graph, i.e., the Laplacian quadratic $Q(\boldsymbol{L})$, is small. This, together with the assumption that the signal follows a GMRF, leads to the following optimization problem:

$$\begin{aligned}
\text{minimize} \quad & \underbrace{\|\mathbf{X}-\mathbf{Y}\|_F^2}_{data\ \ fidelity} + \underbrace{\underbrace{\alpha\ \mathrm{tr}(\mathbf{Y}^\top\mathbf{L}\mathbf{Y})}_{smoothness} + \beta\|\mathbf{L}\|_F^2}_{sparsity} \\
\text{subject to} \quad & \mathrm{tr}(\mathbf{L}) = n, \\
& L_{ij} = L_{ji} \leq 0, \qquad i \neq j, \\
& \mathbf{L}\cdot\mathbf{1} = \mathbf{0} \\
\text{variable} \quad & \mathbf{L}, \mathbf{Y}
\end{aligned} \tag{1.4.11}$$

  in which the Laplacian matrix it is obtained $\boldsymbol{L}$ and a smooth (filtered) version of the data $\boldsymbol{y}$, for each node n=1, 2, ..., N.

  We can say that this optimization problem is not convex (it has no global minimum). But it can be solved using an alternating procedure, where we

first fix $\boldsymbol{Y} = \boldsymbol{X}$, so we find $\boldsymbol{L}$ from our data measurements $\boldsymbol{X}$. We can play with the sparsity (more L_{ij}=0, meaning more disconnected nodes) of our solution using the β parameter. Finally, when $\boldsymbol{L}$ is found, we can solve again, fixing $\boldsymbol{L}$, found and finding $\boldsymbol{Y}$ that will be a filtered version of our data $\boldsymbol{X}$.

1.4.4 Virtual sensor: reconstructing the signal

When there is no physical sensor or a node has to be maintained or repaired over a long period of time, missing value imputation methods based on temporal correlations are ineffective. Therefore, one solution is to replace the measurements with a virtual sensor by reconstructing the signal with neighboring nodes using spatial correlations. There are many techniques to create a virtual sensor. Here, we propose using a machine learning model to reconstruct the signal on a given sensor (target) using neighbouring nodes as features, thereby creating a virtual sensor.

The process works as follows: First, construct the graph in any of the 3 ways described previously. In the case of using data-driven techniques, it is recommended to split the data set into two parts and use the first part of the data set to build the graph. However, if not enough data is available, use all the data to build the graph.

Second, a machine learning model is trained in which the true values are the target sensor y_k, and the independent variables $x_1, \cdots, x_P$ are the neighbour nodes. In this case, take the data and use part for training and part for testing.

The goal is to compare the target (true sensor) values with the estimated values (virtual sensor) of your machine learning model. You can compare the virtual sensor using the 3 graph models (different ways of obtaining the neighborhood of a sensor) and different machine learning models, e.g., multiple linear regression (MLR) and support vector regression (SVR). You can also compare a baseline model in which you build your virtual sensor with all the sensors (full mesh graph).

1.4.5 Description of the data

You have two CSV files. The first, called *Report-1-VirtualSensor-Network-Location.csv*, contains the latitude (Lat) and longitude (Lon) of each node (there are 12 nodes). The first column is the name of the node, and the second column (EOI) is an ID. You have to convert geolocations to distances using the *haversine formula*. Look in Google for Python code to obtain these distances.

The second file, called *Report-1-VirtualSensor-Network-Data.csv*, contains the data. The first column contains a timestamp with hourly measurements (2170 samples, from 1st of June 2017 to 06 of October 2017). Then, you have 12

columns (one per node) with O_3 measurements. Therefore, in this dataset, N=12 and M=2258.

1.4.6 Project realization

The project aims to obtain the graph using each of the three techniques [1]: i) distance-based, ii) GLASSO, and iii) Laplacian. We know that a complete mesh network will have N(N-1)/2=66 undirected links. This is a research activity, so the aim is not only to obtain the graph but to study different configurations given different parameters for each model.

- For the distance-based method, you have two parameters, θ and T (threshold), which control the sparsity of the graph. Play with these values and plot various curves of how many links the model gets as a function of these parameters (e.g., set θ to 3-4 values, and plot as a function of T. Plot a graph as an example.

- You can use the Python library *sklearn* for GLASSO. GLASSO depends on the parameter λ to control the sparsity of the precision matrix, and thus the sparsity of the graph. Plot curves on how many links you get as a function of this parameter. With a very large λ, regularization is promoted, and a disconnected network will be obtained, while with the parameter λ close to 0, the network is fully connected.

- For the Laplacian case, you must program the optimisation problem as described using **cvxpy**. Now, you have two parameters to play with: the parameter α is a smoothness penalization constant and controls the sparsity of the Laplacian and the smoothness of the signal; while the parameter β is an l-1 norm Laplacian penalization constant, and it controls the sparsity of the Laplacian.

  You should note that when β decreases and α increases, the algorithm promotes a low number of edges. The reason is that when β increases, the off-diagonal values of the Laplacian matrix tend to be similar (uniformly distributed) and small, resulting in a large number of edges. When β is small, the opposite is true.

  When α is large, the trace of the quadratic term $\boldsymbol{Y}^\top \boldsymbol{L}\boldsymbol{Y}$ is small, promoting both sparsity and smoothness.

  Try to plot the number of edges as both parameters change; e.g., plot the number of edges as a function of the ratio of β and α (depending on the units, use a log-scale for the ratio).

- Reconstruct the signal using some machine learning approaches, such as MLR, SVR or GP in some target nodes and check the RMSE and R^2 of the virtual sensor for several sensors given the neighborhood for the 3 cases. You can also choose some scenarios by changing parameters of the

graph that will give different neighborhoods, for example, dense network, medium network, sparse network. Compare this with the case where there is a full mesh network with and without regularization.

2 Report 2: Development of a proxy sensor for unregulated pollutants.

Deadline: to be delivered in january 2027 (we will fix the day before Christmas) using "practicals" in RACO. Interview in january.

2.1 Motivation

Black carbon is a major component of fine particulate matter, a potent warming agent in the atmosphere that contributes to regional environmental disruption and accelerates glacier melting. BC appears from incomplete combustion and comes mainly from road traffic, it is present in urban aerosols and is linked to cardiovascular and respiratory diseases. The monitoring of BC is not easy and is not regulated by the European Union (EU) Air Quality Directives. In contrast to regulated pollutants, there is not much affordable equipment to monitor BC, which makes the availability of BC measurements operationally expensive and difficult. The new proposal for a Directive of the EU Parliament on ambient air quality and cleaner air for Europe, published in October 2022, states that introducing additional sampling points for unregulated air pollutants of emerging concern, such as ultrafine particles, black carbon, ammonia or the oxidative potential of particulate matter (PM), will support scientific understanding of their effects on health and the environment.

This supports the need to determine BC concentrations, either by direct or indirect methods, in European urban areas. In recent years, there has been a great interest in using low-cost sensors (LCSs) to measure regulated pollutants such as CO, NO_2, NO, SO_2, O_3, and PM_{10} and $PM_{2.5}$ particles. One way to increase the availability of BC measurements without the need to deploy expensive equipment is the use of virtual sensors. A virtual sensor is defined as a mathematical model that estimates the target phenomenon at a specific location where no physical sensor is available. A proxy is a specific type of virtual sensor that estimates a target pollutant from indirect sensor measurements.

2.2 BC proxy using machine learning

The objective of homework 2 is to build a BC proxy using machine learning tools. The estimation model is non-linear, so, you can use any non-linear model. For this homework, we propose that you compare the proxy results with several

well-known machine learning regression tools: support-vector regression (SVR, with your favorite kernel or with several of them), random forest (RF), Gradient Boosting (GBoost), and a Feed-Forward Neural Network (FFNN).

Optionally, if you have time and want to test more models, you can add a Gaussian Process, a Kernel Ridge Regression or a KNN (you can choose one or several of them). It is mandatory that at least you compare SVR, RF, Gradient Boosting and FFNN.

You can use Python libraries such as scikit-learn that provide solvers for these ML models.

The data consists of a CSV file: "BC-Data-Set.csv", where it can be seen that the first row is a header that describes the data:

- **date:** Timestamp (UTC) for each measurement,
- **BC:** true values of BC concentrations, in μgr/m^3,
- **N_CPC:** ultrafine particle number concentration,
- **PM-10:** sensor measurements, in μgr/m^3,
- **PM-2.5:** sensor measurements, in μgr/m^3,
- **PM-1:** sensor measurements, in μgr/m^3,
- **NO:** sensor measurements, in μgr/m^3,
- **O_3:** sensor measurements, in μgr/m^3,
- **SO_2:** sensor measurements, in μgr/m^3,
- **CO:** sensor measurements, in μgr/m^3,
- **NO:** sensor measurements, in μgr/m^3,
- **NOx:** sensor measurements, in μgr/m^3,
- **TEMP:** temperature sensor, in °C,
- **HUM:** relative humidity sensor, in %.

2.3 Some steps to follow

The first step consists of understanding the data. For that purpose, the best approach is to obtain some statistics (means, correlations, etc.) and to plot several curves to see dependencies in the data (scatter-plots and temporal trends).

Now, the second step is to produce the proxy using the machine learning models and compare results in terms of R^2, MAE, and RMSE.

However, you have too many input parameters, and it is possible that some of the features are not useful at all. A good approach is to regularize the model or use a forward subset selection mechanism (FSS). Either of these two solutions can give good results. Choose your favorite.

a) Using forward or backward subset selection: since it is very time-consuming to do this FSS for every machine learning model (it would be a good practice to see which combination produces better selection of features for each one of the machine learning models), you can do it for one of them (choose your favorite one), and then when you get your best selection, fix it to compare the same feature selection with the rest of the machine learning models.

b) Using regularization: another possibility is to use regularization on the ML models (choose your favorite regularization or the one that you think will work better, or both and compare, e.g., norm l-1 or l-2).

Again, plot results or/and use tables to show your results. Remember to optimize the hyperparameters of the models, and it would be nice if you put some results on how a model evolves depending on the hyperparameters.

One important aspect is the temporal nature of the data. If you mix the data, you break the temporal continuity, and the training set will contain data collected under different environmental conditions, which will result in a relatively good estimate. However, it is common to have data collected over a period of time that does not represent all environmental conditions; for example, you might train in summer but make estimates in winter. This leads to 'concept drift', a phenomenon that is difficult to resolve. Compare the proxy results for both cases: one where the data is shuffled and the seasonality is disrupted, and one where the data is not shuffled. In either case, recover the temporality if you plot the results.

As a summary, plot results, show results, show intermediate values (e.g., results with several features using the subset selection), hyperparameters, etc. Get your conclusions.

References

[1] Ferrer-Cid, Pau, Jose M. Barceló-Ordinas, and Jorge García-Vidal. *Graph learning techniques using structured data for IoT air pollution monitoring platforms.* IEEE Internet of Things Journal, 2021, vol. 8, no 17, pp. 13652-13663, https://upcommons.upc.edu/bitstream/handle/2117/345458/VERSION-DRAC.pdf?sequence=1.

[2] J. Friedman, T. Hastie, and R. Tibshirani, *Sparse inverse covariance estimation with the graphical lasso*, Biostatistics, vol. 9, no. 3, pp. 432- 441, 2008 https://arxiv.org/abs/0708.3517.

[3] X. Dong, D. Thanou, P. Frossard and P. Vandergheynst, *Laplacian matrix learning for smooth graph signal representation* 2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), South Brisbane, QLD, Australia, 2015, pp. 3736-3740, https://arxiv.org/pdf/1406.7842.pdf.